



# ANGULAR

Data Binding & Signal & Lifecycle

Angular

Séance 2 / 5

Module : Développement Full Stack Moderne

Filière : ILISI1

Standalone  
Components

Les 4 types de  
Data Binding

Communication  
Input / Output

Lifecycle  
Hooks

Pr. CHAKRI Sana

2025/2026

# Plan de la Séance 2

Ce que vous allez apprendre aujourd'hui

## 01 Rappel Architecture

Structure projet Angular 20, composants standalone

## 02 Standalone Components

Composants autonomes, imports directs, avantages

## 03 Data Binding

Interpolation, Property, Event et Two-Way binding

## 04 Input & Output

Communication parent → enfant et enfant → parent

## 05 Lifecycle Hooks

ngOnInit, ngOnDestroy, ngOnChanges et les autres

## 06 TP — Gestion Contacts

Appli complète : formulaire + liste + service

# Objectifs de la séance 2

*À la fin de cette séance, vous serez capables de :*



Créer et utiliser des composants standalone Angular



Utiliser les 4 types de data binding : interpolation, property, event et two-way (ngModel)



Faire communiquer un composant parent et un composant enfant via Input() et Output()



Comprendre et utiliser les lifecycle hooks, notamment ngOnInit et ngOnDestroy



Construire une application de gestion de contacts avec des composants imbriqués

# 🔗 Composants Standalone — La nouvelle approche Angular

💡 **Définition :** : Un composant standalone fonctionne **INDÉPENDAMMENT**, sans NgModule. Depuis Angular 17 c'était le mode par défaut. En Angular 21, standalone est **IMPLICITE** — plus besoin d'écrire **standalone: true**, c'est la valeur par défaut !

## Avant (Angular < 14) — Avec NgModule

```
// app.module.ts (plus nécessaire !)  
@NgModule({  
  declarations: [MonComponent],  
  imports: [BrowserModule],  
})  
export class AppModule { }  
  
// composant.ts  
@Component({  
  selector: 'app-mon-comp',  
  templateUrl: './mon-comp.html'  
})  
export class MonComponent { }
```



## Maintenant (Angular 21) — Standalone

```
// Plus besoin de NgModule !  
// Plus besoin de standalone: true (v21)  
  
@Component({  
  selector: 'app-mon-comp',  
  imports: [FormsModule, CommonModule],  
  // standalone: true → implicite en v21  
  templateUrl: './mon-comp.html'  
})  
export class MonComponent { }
```

# ? Standalone — Avantages & Comment l'utiliser

## ✓ Indépendance totale

Un composant standalone peut être utilisé directement dans n'importe quel autre composant sans passer par un module.

## 🔗 Imports directs

Chaque composant déclare lui-même ce dont il a besoin dans son tableau imports. Plus clair, plus lisible.

## 🔄 Réutilisabilité facile

Un composant standalone peut être partagé entre projets sans aucune configuration de module supplémentaire.

## 🚀 Code allégé

Moins de configuration : plus de fichiers app.module.ts.

```
// Utiliser un composant standalone dans un autre
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { ProduitCardComponent } from '../produit-card/produit-card.component';

@Component({
  selector: 'app-catalogue',
  // standalone: true → implicite en v21
  // Déclarez ici tout ce que vous utilisez dans le template :
  imports: [
    CommonModule,           // @If, @For, pipes built-in...
    FormsModule,            // [(ngModel)] pour two-way binding
    ProduitCardComponent    // Votre propre composant standalone
  ],
  templateUrl: './catalogue.component.html',
  styleUrls: ['./catalogue.component.html'],
})
export class CatalogueComponent { }
```

# 👁 Le Data Binding — Vue d'ensemble



## Définition :

Le data binding est le mécanisme qui synchronise les données entre le composant TypeScript (.ts) et son template HTML (.html). Quand une donnée change dans le composant, la vue se met à jour, et vice versa.

1

### Interpolation

```
{{ expression }}
```

→ Composant → Vue

Affiche une valeur du composant dans le HTML. La valeur s'affiche en tant que texte.

2

### Property Binding

```
[propriete]="val"
```

→ Composant → Vue

Lie une propriété d'élément HTML (src, disabled, href...) à une variable du composant.

3

### Event Binding

```
(evenement)="fn()"
```

← Vue → Composant

Écoute un événement utilisateur (clic, frappe...) et appelle une méthode du composant.

4

### Two-Way Binding

```
[(ngModel)]="prop"
```

↔ Composant ↔ Vue

Synchronisation bidirectionnelle. Utilisé pour les formulaires. Nécessite FormsModule.

# 👁 Data Binding 1/4 — Interpolation {{ expression }}

Sens : Composant TypeScript → Template HTML | Utilisé pour : afficher une valeur ou le résultat d'une expression dans le HTML

```
// composant.ts
export class AppComponent {
  prenom: string = 'Ahmed';
  score: number = 18;
  notes: number[] = [14, 16, 18, 12];
}
```

```
<!-- template.html -->
<h1>Bonjour {{ prenom }} !</h1>

<!-- Expressions JS autorisées -->
<p>Score : {{ score }}</p>
<p>Score / 20 : {{ score / 20 }}</p>
<p>{{ prenom.toUpperCase() }}</p>
<p>Notes : {{ notes.length }} notes</p>
<p>Réussite : {{ score >= 10 ? '✔' : '✗' }}</p>
```

## Règles importantes :

✔ **Autorisé**    {{ nom }} • {{ a + b }} • {{ arr.length }} • {{ fn() }}

✔ **Autorisé**    {{ condition ? 'oui' : 'non' }} • {{ nom.toUpperCase() }}

✗ **Interdit**    Assignments : {{ a = 5 }} • new, typeof, instanceof

✗ **Interdit**    Blocs if/for/while • Incréments : {{ a++ }}

## 📄 Résultat dans le navigateur

**Bonjour Ahmed !**

Score : 18 — Score / 20 : 0.9

AHMED — Notes : 4 notes — Réussite : ✔

## 👁 Data Binding 2/4 — Property Binding [propriété]="valeur"

Sens : Composant TypeScript → Template HTML | Utilisé pour : lier une propriété d'un élément DOM à une variable du composant

```
// composant.ts
export class AppComponent {
  imageUrl: string = 'https://picsum.photos/200/100';
  boutonActif: boolean = false;
  lienSite: string = 'https://angular.dev';
  tailleTexte: number = 20;
  classeCSS: string = 'alerte-rouge';
}
```

### ⚠ Interpolation vs Property Binding — Quelle différence ?

src="{{ imageUrl }}" → [src]="imageUrl"

Pour src d'image : les deux fonctionnent, [src] est préféré

disabled="{{ isDisabled }}" → [disabled]="isDisabled"

Pour booléen : utiliser OBLIGATOIREMENT [disabled]

```
<!-- template.html -->

<!-- Lier src d'une image -->
<img [src]="imageUrl" alt="Photo">

<!-- Désactiver un bouton dynamiquement -->
<button [disabled]="!boutonActif">Envoyer</button>

<!-- Lier href d'un lien -->
<a [href]="lienSite">Site Angular</a>

<!-- Lier un style inline -->
<p [style.fontSize.px]="tailleTexte">Texte</p>

<!-- Lier une classe CSS -->
<div [class]="classeCSS">Alerte !</div>
```



## 🔗 Data Binding 3/4 — Event Binding (événement)="méthode()"

Sens : Template HTML → Composant TypeScript | Utilisé pour : réagir aux actions de l'utilisateur (clic, frappe, survol...)

```
// composant.ts
export class AppComponent {
  compteur: number = 0;
  texte: string = '';
  couleur: string = 'blanc';

  incrementer(): void {
    this.compteur++;
  }

  //Met à jour la propriété `texte` avec la valeur saisie.
  onFrappe(event: Event): void {
    // Conversion explicite de la cible de l'événement en élément input HTML
    const input = event.target as HTMLInputElement;
    this.texte = input.value;
  }

  changerCouleur(c: string): void {
    this.couleur = c;
  }
}
```

```
<!-- template.html -->

<!-- Clic simple -->
<button (click)="incrementer()">
  Clics : {{ compteur }}
</button>

<!-- Passer un paramètre -->
<button (click)="changerCouleur('rouge')">Rouge</button>
<button (click)="changerCouleur('bleu')">Bleu</button>

<!-- Récupérer l'événement avec $event -->
<input (keyup)="onFrappe($event)">
<p>Vous tapez : {{ texte }}</p>

<!-- Autres événements courants -->
<!-- (mouseover) (mouseout) (change) (submit) -->
<!-- (focus) (blur) (dblclick) (keydown) -->
```

## ☞ Data Binding 4/4 — Two-Way Binding [(ngModel)]="propriété"

Sens : Composant ↔ Template (les deux) | Quand la variable change → l'input se met à jour. Quand l'utilisateur tape → la variable se met à jour.

### ⚠ Prérequis :

Pour utiliser [(ngModel)], vous devez importer FormsModule dans le tableau imports de votre composant standalone : imports: [FormsModule]

```
// composant.ts
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  standalone: true,
  imports: [FormsModule], // ← Obligatoire !
  templateUrl: './...'
})
export class AppComponent {
  prenom: string = '';
  age: number = 0;
}
```

```
<!-- template.html -->
<input [(ngModel)]="prenom"
       placeholder="Votre prénom">

<input [(ngModel)]="age"
       type="number"
       placeholder="Votre âge">

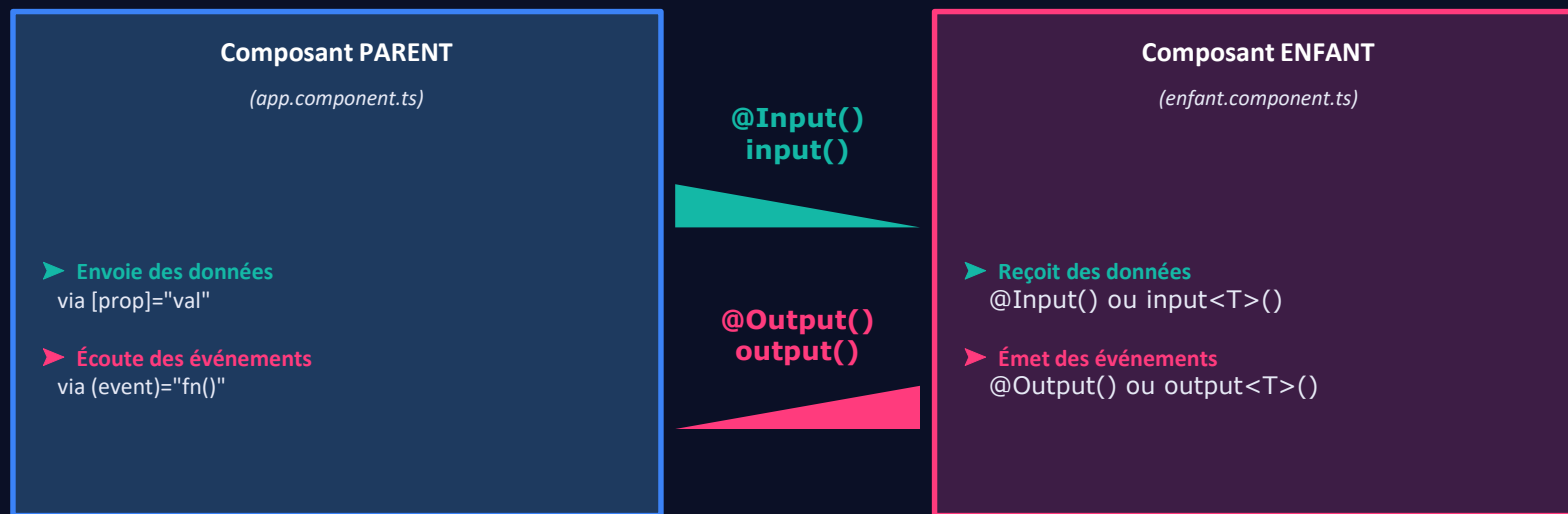
<!-- Affichage en temps réel -->
<p>Bonjour {{ prenom }},
    vous avez {{ age }} ans !</p>
```

# ∞ Récapitulatif — Les 4 types de Data Binding

| Type                | Syntaxe                      | Sens      | Cas d'usage                          | Import requis      |
|---------------------|------------------------------|-----------|--------------------------------------|--------------------|
| 1. Interpolation    | <code>{{ valeur }}</code>    | TS → HTML | Afficher du texte, des calculs       | Aucun              |
| 2. Property Binding | <code>[src]="url"</code>     | TS → HTML | Propriétés DOM : src, disabled, href | Aucun              |
| 3. Event Binding    | <code>(click)="fn()"</code>  | HTML → TS | Réagir aux actions utilisateur       | Aucun              |
| 4. Two-Way Binding  | <code>[(ngModel)]="p"</code> | TS ↔ HTML | Formulaires, champs de saisie        | <b>FormsModule</b> |

```
<!-- Exemples combinés dans un même template -->
<h1>{{ titre }}</h1>                                <!-- Interpolation -->
<img [src]="urlPhoto" [alt]="nomUtilisateur">        <!-- Property Binding -->
<input [(ngModel)]="recherche">                       <!-- Two-Way Binding -->
<button (click)="lancerRecherche()">Go</button>        <!-- Event Binding -->
```

Dans Angular, les composants forment une hiérarchie parent-enfant. Pour qu'ils échangent des données, Angular fournit **deux approches** : l'approche classique avec décorateurs `@Input()` / `@Output()` et la nouvelle approche basée sur les signals `input()` / `output()`



↓ **@Input() / input() — Parent → Enfant**  
Le parent envoie une donnée à l'enfant via `[propiete]="valeur"`

↑ **@Output() / output() — Enfant → Parent**  
L'enfant notifie le parent d'un événement via `emit(value)`

Angular 21 recommande la nouvelle API **input()** : plus concise, sans décorateur, intégrée aux signals.

## ◆ Classique — @Input() décorateur

```
// carte-produit.component.ts
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-carte-produit',
  template: `
    <div>
      <h3>{{ nom }}</h3>
      <p>Prix : {{ prix }} MAD</p>
    </div>
  `
})
export class CarteProduitComponent {
  @Input() nom: string = '';
  @Input() prix: number = 0;
}
```

## » Angular 21 — input() signal

```
// input-model.component.ts
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-input-model',
  template: `
    <div>
      <h3>{{ nom() }}</h3>
      <p>Prix : {{ prix() }} MAD</p>
    </div>
  `
})
export class InputModelComponent {
  nom = input<string>(""); // InputSignal<string>
  prix = input.required<number>(); // obligatoire
}
```

```
<h1>Input signals</h1>
<app-input-model [prix]=prix nom={{nom}} ></app-input-model>
```

----

```
export class App {
  prix=22;
  nom='Anouar';
}
```

La fonction **output()** remplace **@Output()** + **new EventEmitter()** : plus concis, même sémantique. Le parent écoute toujours avec **(event)**.

### ◆ Classique — @Output() + EventEmitter

```
// ajout-contact.component.ts
import { Component, Output,
  EventEmitter } from '@angular/core';

@Component({ ... })
export class AjoutContactComponent {
  nom: string = "";

  @Output() contactAjoute =
    new EventEmitter<string>();

  envoyer(): void {
    this.contactAjoute.emit(this.nom);
    this.nom = "";
  }
}
```

```
<br><br>
<h1>Output signals</h1>
<app-output-signal (contactAjoute)="traiterNom($event)" />
<p>Nom reçu : {{ nomRecu() }}</p>
```

----

```
nomRecu=signal<string>('');
traiterNom(valeur: string): void {
  this.nomRecu.set(valeur);
  valeur='';
  console.log('Valeur reçue de l\'enfant :', valeur);
}
```

### » Angular 21 — output() signal

```
// output-signal.component.ts
import { Component, output } from '@angular/core';

@Component({
  selector: 'app-output-signal',
  template: `
    <h3>Enfant Component (OUTPUT)</h3>
    <input
      type="text"
      [(ngModel)]="unNom"
      placeholder="Entrez un nom"
    />
    <button (click)="envoyer()">Envoyer au parent</button> `
})
export class OutputSignalComponent {
  nom : string = "";

  contactAjoute = output<string>();

  envoyer(): void {
    this.contactAjoute.emit(this.nom);
    this.nom = "";
  }
}
```

★ **Définition** : Angular crée, met à jour et détruit les composants automatiquement. Les **lifecycle hooks** sont des méthodes spéciales appelées automatiquement au bon moment. En v21, `effect()` offre une alternative réactive pour la plupart des cas.

1

## constructor()

*Naissance*

Injection des services. Ne pas charger de données ici.

2

## ngOnChanges()

*@Input change*

Appelé chaque fois qu'un `@Input()` reçoit une nouvelle valeur.

3

## ngOnInit() ★

*Initialisation*

LE PLUS IMPORTANT. Charger données, appels HTTP. Appelé UNE fois.

4

## ngAfterViewInit()

*Vue rendue*

La vue HTML est construite. Idéal pour accéder au DOM.

5

## ngOnDestroy()

*Destruction*

Composant retiré du DOM. Se désabonner des Observables.

**ngOnInit() ★ Le plus important**

```
import { Component, OnInit } from '@angular/core';

@Component({ ... })
export class MonComponent implements OnInit {

  produits: string[] = [];
  titre: string = "";

  // constructor : UNIQUEMENT injection
  constructor(private produitService: ProduitService) {}

  // ngOnInit : pour tout le reste !
  ngOnInit(): void {
    // ► Charger les données
    this.produits = this.produitService
      .getProduits();
    // ► Initialiser variables
    this.titre = 'Catalogue — ' +
      new Date().getFullYear();
  }
}
```

**ngOnDestroy() Nettoyage obligatoire**

```
import { Component, OnInit,
  OnDestroy } from '@angular/core';
import { Subscription } from 'rxjs';

export class MonComponent
  implements OnInit, OnDestroy {

  private subscription!: Subscription;

  ngOnInit(): void {
    this.subscription =
      this.service.getData()
        .subscribe(data => {
          // traitement...
        });
  }

  // Appelé avant destruction
  ngOnDestroy(): void {
    // ► Éviter fuites mémoire
    this.subscription.unsubscribe();
  }
}
```



**ngOnChanges()** est appelé automatiquement à CHAQUE FOIS qu'un @Input() reçoit une nouvelle valeur. C'est le seul hook qui reçoit des paramètres (SimpleChanges).

```
import { Component, Input, OnChanges,
        SimpleChanges } from '@angular/core';

@Component({
  selector: 'app-afficheur',
  template: `<p>Valeur actuelle : {{ valeur }}</p>`
})
export class AfficheurComponent implements OnChanges {

  @Input() valeur: number = 0;

  // Appelé à CHAQUE changement d'un @Input()
  ngOnChanges(changes: SimpleChanges): void {
    if (changes['valeur']) {
      const ancienne = changes['valeur'].previousValue;
      const nouvelle = changes['valeur'].currentValue;
      console.log(`Avant : ${ancienne} → Après : ${nouvelle}`);
    }
  }
}
```

### Quand est-il appelé ?

- , Avant ngOnInit (1er appel)
- , À chaque changement d'@Input()

### Propriétés SimpleChanges

#### .currentValue

Nouvelle valeur

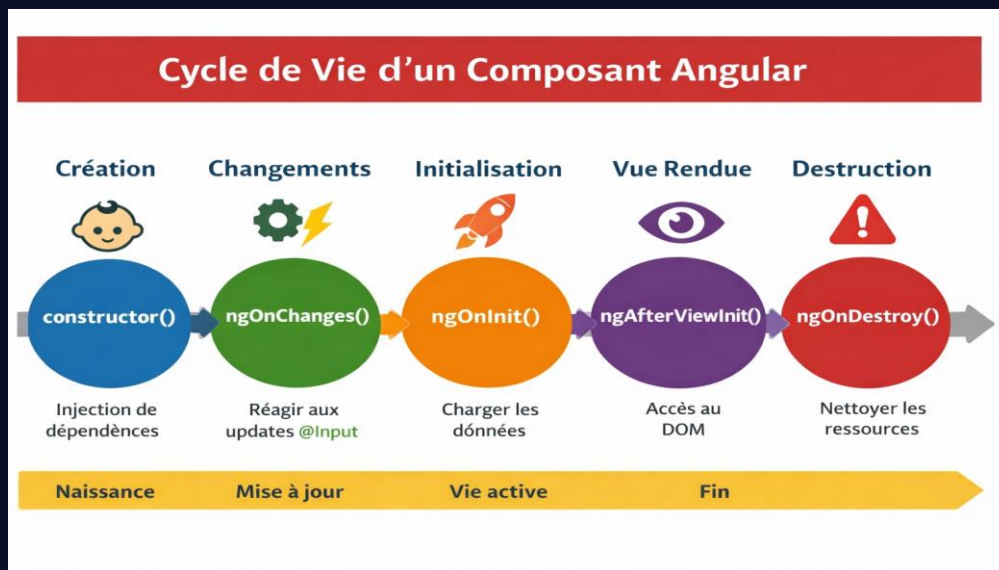
#### .previousValue

Ancienne valeur

#### .firstChange

Premier changement ?

| Hook              | Interface     | Quand ?                | Usage typique                         | Fréquence |
|-------------------|---------------|------------------------|---------------------------------------|-----------|
| constructor()     | —             | À la création          | Injection de dépendances UNIQUEMENT   | 1 fois    |
| ngOnChanges()     | OnChanges     | @Input reçoit valeur   | Réagir aux nouvelles valeurs d'entrée | N fois    |
| ngOnInit() ★      | OnInit        | Après le 1er affichage | Charger données, init, HTTP           | 1 fois    |
| ngAfterViewInit() | AfterViewInit | Vue HTML rendue        | Accéder DOM (viewChild signal)        | 1 fois    |
| ngOnDestroy()     | OnDestroy     | Avant destruction      | Unsubscribe, libérer ressources       | 1 fois    |





# TRAVAUX PRATIQUES

Séance 2 — Gestion de Contacts avec Data Binding

**Durée : 2h00**

Objectif : créer un formulaire d'ajout de contact avec aperçu en temps réel

```
// formulaire-contact.component.ts
import { Component, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-formulaire-contact',
  imports: [FormsModule],
  templateUrl: './formulaire-contact.component.html'
})
export class FormulaireContactComponent {
  // Angular 21 : signals pour l'état local
  nom = signal("");
  email = signal("");
  telephone = signal("");
}
```

```
<!-- formulaire-contact.component.html -->
<div>
  <h2>Ajouter un contact</h2>

  <!-- signals + ngModel -->
  <input [ngModel]="nom()"
    (ngModelChange)="nom.set($event)"
    name="nom" placeholder="Nom complet">

  <input [ngModel]="email()"
    (ngModelChange)="email.set($event)"
    name="email" type="email"
    placeholder="Email">

  <input [ngModel]="telephone()"
    (ngModelChange)="telephone.set($event)"
    name="tel" placeholder="Téléphone">

  <!-- Aperçu en temps réel -->
  <div>
    <h3>Aperçu :</h3>
    <p><b>{{ nom() }}</b></p>
    <p>{{ email() }} | {{ telephone() }}</p>
  </div>
</div>
```

Objectif : ajouter un bouton 'Sauvegarder' qui envoie le contact au composant parent via output()

```
// formulaire-contact.component.ts — māj
import { output } from '@angular/core';

export interface Contact {
  nom: string; email: string; telephone: string;
}

export class FormulaireContactComponent {
  nom = signal(""); email = signal("");
  telephone = signal("");

  // v21 : output() au lieu d'EventEmitter
  contactSauvegarde = output<Contact>();

  sauvegarder() {
    if (this.nom() && this.email()) {
      this.contactSauvegarde.emit({
        nom: this.nom(),
        email: this.email(),
        telephone: this.telephone()
      });
      this.nom.set(""); this.email.set("");
    }
  }
}
```

```
// app.component.ts
import { Component, signal } from '@angular/core';
import { FormulaireContactComponent,
  Contact } from './formulaire-contact/...';
import { ListeContactsComponent } from './liste-contacts/...';

@Component({
  imports: [FormulaireContactComponent,
    ListeContactsComponent],
  template: `
    <app-formulaire-contact
      (contactSauvegarde)="ajouter($event)">
    </app-formulaire-contact>

    <app-liste-contacts
      [contacts]="contacts()">
    </app-liste-contacts>
  `
})
export class AppComponent {
  contacts = signal<Contact[]>([]);

  ajouter(c: Contact) {
    this.contacts.update(arr => [...arr, c]);
  }
}
```

```
// liste-contacts.component.ts
import { Component, Input, Effect, OnDestroy } from '@angular/core';

@Component({
  selector: 'app-liste-contacts',
  template: `
    <h2>Contacts {{ contacts().length }}</h2>
    @if (contacts().length === 0) {
      <p>Aucun contact.</p>
    }
    <ul>
      @for (c of contacts()); track c.email;
      let i = $index {
        <li>{{ i + 1 }}. {{ c.nom }} — {{ c.email }}</li>
      }
    </ul>
  `
})
export class ListeContactsComponent implements OnDestroy {

  contacts = input<Contact[]>([]);

  constructor() {
    // effect() : remplace ngOnInit+ngOnChanges
    effect(() => {
      console.log(` ${this.contacts().length} contact(s)`);
    });
  }

  ngOnDestroy() {
    console.log('Liste détruite — nettoyage');
  }
}
```

```
Ajouter 2-3 contacts initiaux dans constructor :
this.contacts.set([
  {nom:'Ali', email:'ali@test.com', ...},
  {nom:'Sara', email:'sara@test.com', ...}
]);
```

À chaque changement de contacts(), le message s'affiche automatiquement dans la console.

F12 → Console. Vous verrez effect() et ngOnDestroy au bon moment.

## Architecture de l'application Angular 21 :

### AppComponent (composant racine)

```
contacts = signal<Contact[]>([])
ajouter(c: Contact) { ... update ... }
```



[contacts]

### FormulaireContactComponent

```
output<Contact>() contactSauvegarde
signal("") pour chaque champ
Bouton (click)="sauvegarder()"
[ngModel] + (ngModelChange)
```



(contactSauvegarde)

### ListeContactsComponent

```
input<Contact[]>([]) contacts
@for pour afficher la liste
@if pour 'Aucun contact'
effect() pour logger les changements
ngOnDestroy() pour nettoyage
```

► **Commandes CLI :** ng g c formulaire-contact | ng g c liste-contacts | ng serve *(standalone implicite en v21)*

## formulaire-contact.component.html

```
<div class="formulaire">
  <h2>Ajouter un Contact</h2>

  <input [ngModel]="nom()"
    (ngModelChange)="nom.set($event)"
    name="nom"
    placeholder="Nom complet">
  <input [ngModel]="email()"
    (ngModelChange)="email.set($event)"
    name="email" type="email"
    placeholder="Email">
  <input [ngModel]="telephone()"
    (ngModelChange)="telephone.set($event)"
    name="tel"
    placeholder="Téléphone">

  <button (click)="sauvegarder()">
    + Ajouter
  </button>

  @if (nom()) {
    <div class="aperçu">
      <b>Aperçu :</b> {{ nom() }} — {{ email() }}
    </div>
  }
</div>
```

## liste-contacts.component.html

```
<div class="liste">
  <h2>Liste des Contacts
    ({{ contacts().length }})</h2>

  @if (contacts().length === 0) {
    <p>Aucun contact pour le moment.</p>
  } @else {
    <ul>
      @for (c of contacts();
        track c.email;
        let i = $index) {
        <li class="contact-item">
          <span class="num">{{ i + 1 }}</span>
          <strong>{{ c.nom }}</strong>
          <span>{{ c.email }}</span>
          <small>{{ c.telephone }}</small>
        </li>
      }
    </ul>
  }
</div>
```





**Bon courage pour le TP !**

Prochaine séance : Directives, Pipes & Introduction aux Services

Angular 21 | Séance 2 | 2025/2026

**Des questions ?**